

Chapter 6

Searching Algorithms and Hashing

Sequential Search / Linear Search

Linear search is a sequential searching algorithm where we start from one end and check every element of the list until the desired element is found. It is the simplest searching algorithm.

In Linear Search, we iterate over all the elements of the array and check if it the current element is equal to the target element. If we find any element to be equal to the target element, then return the index of the current element. Otherwise, if no element is equal to the target element, then return -1 as the element is not found. Linear search is also known as **sequential search**.

Algorithm

1. Start
2. Read the search element from the user.
3. Compare, the search element with the first element in the list.
4. If both are matching, the display "Given element found" and terminate the function.
5. If both are not matching, then compare search element with the next element in the list.
6. Repeat steps 4 and 5 until the search element is compared with the last element in the list.
7. If the last element in the list is also doesn't match, then display "Element not found" and terminate.
8. Stop

How Linear Search Works?

The following steps are followed to search for an element $k = 1$ in the list below.



Array to be searched for

1. Start from the first element, compare k with each element x .

$k = 1$



$k \neq 2$



$k \neq 4$



$k \neq 0$

Compare with each element

2. If $x == k$, return the index.



$k = 1$

Element found

3. Else, return `not found`.

What are some real-world applications of linear search algorithm?

Linear search algorithm can be used in scenarios such as searching for a specific value in a phone book, searching for a name in an unsorted list of contacts, or finding an item in a grocery list.

Time and Space Complexity of Linear Search

Algorithm:

Time Complexity:

- **Best Case:** In the best case, the key might be present at the first index. So the best case complexity is $O(1)$
- **Worst Case:** In the worst case, the key might be present at the last index i.e., opposite to the end from which the search has started in the list. So the worst-case complexity is $O(N)$ where N is the size of the list.
- **Average Case:** $O(N)$

Auxiliary Space: $O(1)$ as except for the variable to iterate through the list, no other variable is used.

Binary Search

Binary search is a search algorithm used to find the position of a target value within a **sorted** array. It works by repeatedly dividing the search interval in half until the target value is found or the interval is empty. The search interval is halved by comparing the target element with the middle value of the search space.

Conditions to apply Binary Search Algorithm in a Data Structure

To apply Binary Search algorithm:

- The data structure must be sorted.
- Access to any element of the data structure should take constant time.

How does Binary Search work?

Binary Search compares the target value to the middle element of the array. If they are equal, the search is successful. If the target is less than the middle element, the search continues in the lower half of the array. If the target is greater, the search continues in the upper half. This process repeats until the target is found or the search interval is empty.

Note: Binary search can be implemented only on a sorted list of items. If the elements are not sorted already, we need to sort them first.

Algorithm

Step 1 – Select the middle item in the array and compare it with the key value to be searched. If it is matched, return the position of the median.

Step 2 – If it does not match the key value, check if the key value is either greater than or less than the median value.

Step 3 – If the key is greater, perform the search in the right sub-array; but if the key is lower than the median value, perform the search in the left sub-array.

Step 4 – Repeat steps 1,2 and 4 iteratively until the size of sub-array becomes 1.

Step 5 – If the key value doesnot exist in the array, then the algorithm returns an unsuccessful search.

Working of Binary search

Now, let's see the working of the Binary Search Algorithm.

To understand the working of the Binary search algorithm, let's take a sorted array. It will be easy to understand the working of Binary search with an example.

There are two methods to implement the binary search algorithm -

- Iterative method
- Recursive method

The recursive method of binary search follows the divide and conquer approach.

Let the elements of array are -

0	1	2	3	4	5	6	7	8
10	12	24	29	39	40	51	56	69

Let the element to search is, **K = 56**

We have to use the below formula to calculate the **mid** of the array -

1. $\text{mid} = (\text{beg} + \text{end})/2$

So, in the given array -

beg = 0

end = 8

mid = $(0 + 8)/2 = 4$. So, 4 is the mid of the array.

0	1	2	3	4	5	6	7	8
10	12	24	29	39	40	51	56	69



$A[mid] = 39$
 $A[mid] < K$ (or, $39 < 56$)
 So, $beg = mid + 1 = 5$, $end = 8$
 Now, $mid = (beg + end)/2 = 13/2 = 6$

0	1	2	3	4	5	6	7	8
10	12	24	29	39	40	51	56	69



$A[mid] = 51$
 $A[mid] < K$ (or, $51 < 56$)
 So, $beg = mid + 1 = 7$, $end = 8$
 Now, $mid = (beg + end)/2 = 15/2 = 7$

0	1	2	3	4	5	6	7	8
10	12	24	29	39	40	51	56	69



$A[mid] = 56$
 $A[mid] = K$ (or, $56 = 56$)
 So, $location = mid$
 Element found at 7th location of the array

Now, the element to search is found. So algorithm will return the index of the element matched.

Time Complexity

Best Case

$O(1)$

Average Case

$O(\log n)$

Worst Case

$O(\log n)$

- **Best Case Complexity** - In Binary search, best case occurs when the element to search is found in first comparison, i.e., when the first middle element itself is the element to be searched. The best-case time complexity of Binary search is **$O(1)$** .
 - **Average Case Complexity** - The average case time complexity of Binary search is **$O(\log n)$** .
 - **Worst Case Complexity** - In Binary search, the worst case occurs, when we have to keep reducing the search space till it has only one element. The worst-case time complexity of Binary search is **$O(\log n)$** .
-

Space Complexity

Space Complexity

$O(1)$

- The space complexity of binary search is $O(1)$.
-

Application of Binary Search

Binary search is a popular algorithm used to find elements in sorted lists. It has many applications, including searching for words in a dictionary, finding phone numbers in a phone book, and finding the best match for a stock price.

Hashing

Hashing is the process of transforming any given key or a string of characters into another value. This is usually represented by a shorter, fixed-length value or key that represents and makes it easier to find or employ the original string.

Hashing is commonly used to create a unique identifier for a piece of data, which can be used to quickly look up that data in a large dataset. For

example, a web browser may use hashing to store website passwords securely. When a user enters their password, the browser converts it into a hash value and compares it to the stored hash value to authenticate the user.

There are numerous hashing algorithms, each with distinct advantages and disadvantages. The most popular algorithms include the following:

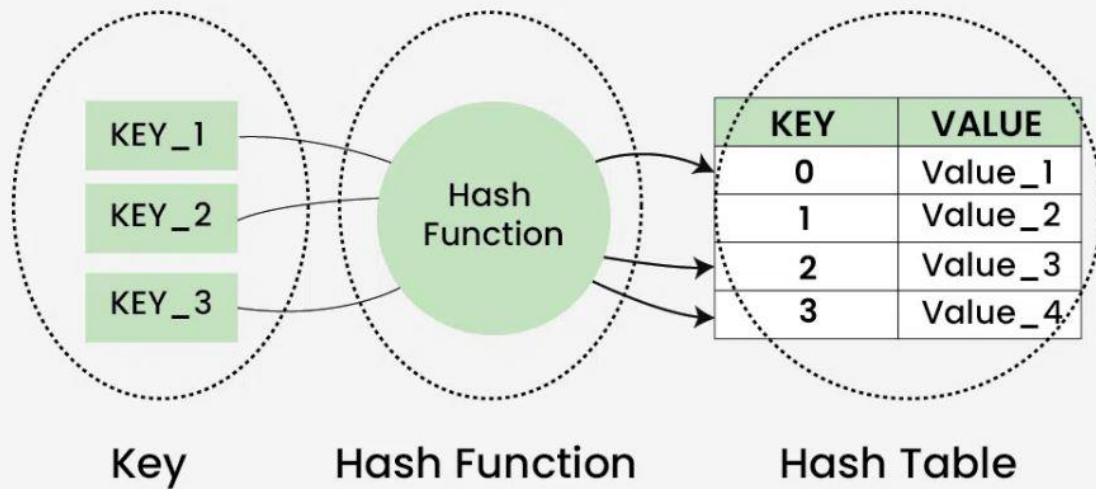
- **MD5:** A widely used hashing algorithm that produces a 128-bit hash value.
 - **SHA-1:** A popular hashing algorithm that produces a 160-bit hash value.
 - **SHA-256:** A more secure hashing algorithm that produces a 256-bit hash value.
-

Components of Hashing

There are majorly three components of hashing:

1. **Key:** A Key can be anything string or integer which is fed as input in the hash function the technique that determines an index or location for storage of an item in a data structure.
2. **Hash Function:** Receives the input key and returns the index of an element in an array called a hash table. The index is known as the hash index .
3. **Hash Table:** Hash table is typically an array of lists. It stores values corresponding to the keys. Hash stores the data in an associative manner in an array where each data value has its own unique index.

Components of Hashing



Hash Function:

Hash Function: A hash function is a type of mathematical operation that takes an input (or key) and outputs a fixed-size result known as a hash code or hash value. The hash function must always yield the same hash code for the same input in order to be deterministic. Additionally, the hash function should produce a unique hash code for each input, which is known as the hash property.

There are different types of hash functions, including:

- **Division method:**

This method involves dividing the key by the table size and taking the remainder as the hash value. For example, if the table size is 10 and the key is 23, the hash value would be 3 ($23 \% 10 = 3$).

- **Multiplication method:**

This method involves multiplying the key by a constant and taking the fractional part of the product as the hash value. For example, if the key is 23 and the constant is 0.618, the hash value would be 2 ($\text{floor}(10 * (0.61823 - \text{floor}(0.61823))) = \text{floor}(2.236) = 2$).

- **Universal hashing:**

This method involves using a random hash function from a family of hash functions. This ensures that the hash function is not biased towards any particular input and is resistant to attacks.

How does Hashing work?

Suppose we have a set of strings {"ab", "cd", "efg"} and we would like to store it in a table.

- **Step 1:** We know that hash functions (which is some mathematical formula) are used to calculate the hash value which acts as the index of the data structure where the value will be stored.
- **Step 2:** So, let's assign
 - "a" = 1,
 - "b" = 2, .. etc, to all alphabetical characters.
- **Step 3:** Therefore, the numerical value by summation of all characters of the string:
 - "ab" = 1 + 2 = 3,
 - "cd" = 3 + 4 = 7,
 - "efg" = 5 + 6 + 7 = 18
- **Step 4:** Now, assume that we have a table of size 7 to store these strings. The hash function that is used here is the sum of the characters in **key mod Table size**. We can compute the location of the string in the array by taking the **sum(string) mod 7**.
- **Step 5:** So we will then store
 - "ab" in $3 \bmod 7 = 3$,
 - "cd" in $7 \bmod 7 = 0$, and
 - "efg" in $18 \bmod 7 = 4$.

Mapping Key with indices of Array

0	1	2	3	4	5	6
cd			ab	egf		

The above technique enables us to calculate the location of a given string by using a simple hash function and rapidly find the value that is stored in that location. Therefore the idea of hashing seems like a great way to store (key, value) pairs of the data in a table.

Hash Table

Hash table is one of the most important data structures that uses a special function known as a hash function that maps a given value with a key to access the elements faster.

A Hash table is a data structure that stores some information, and the information has basically two main components, i.e., key and value. The hash table can be implemented with the help of an associative array. The efficiency of mapping depends upon the efficiency of the hash function used for mapping.

For example, suppose the key value is John and the value is the phone number, so when we pass the key value in the hash function shown as below:

$\text{Hash}(\text{key}) = \text{index};$

When we pass the key in the hash function, then it gives the index.

$\text{Hash}(\text{john}) = 3;$

The above example adds the john at the index 3.

Drawback of Hash function

A Hash function assigns each value with a unique key. Sometimes hash table uses an imperfect hash function that causes a collision because the hash function generates the same key of two different values.

Hashing in Data Structure

- **Hashing** is a technique used in data structures that efficiently stores and retrieves data in a way that allows for quick access.

- Hashing involves mapping data to a specific index in a hash table (an array of items) using a **hash function** that enables fast retrieval of information based on its key.
- The great thing about hashing is, we can achieve all three operations (search, insert and delete) in $O(1)$ time on average.
- Hashing is mainly used to implement a set of distinct items and dictionaries (key value pairs).

Hashing is considered a **searching technique** because it enables efficient retrieval of data using a key. The idea behind hashing is to map keys to specific locations in a data structure (usually an array) using a **hash function**, allowing constant-time complexity $O(1)$ for search operations in the average case.

Here's how it works:

1. **Hash Function:** A hash function takes an input (key) and computes an index in an array (hash table). This index determines where the corresponding value should be stored or searched.
2. **Direct Access:** When you search for a key, the hash function computes the index, and the algorithm directly accesses that index in the hash table to retrieve the value. This eliminates the need to iterate through the entire data structure.
3. **Collision Handling:** If two keys produce the same hash (collision), methods like **chaining** or **open addressing** are used to manage and still enable retrieval of the correct value.

Why It's a Searching Technique:

- **Efficient Lookups:** Hashing is one of the fastest searching techniques, with $O(1)$ average time complexity.
- **Key-Based Access:** Unlike linear or binary search, which relies on sequential or sorted data, hashing provides direct access to elements using a key.
- **Applications:** It's widely used in databases, symbol tables, and caching mechanisms where fast searching is critical.

For example, in a hash table of students' records where keys are student IDs, you can directly find a student's record by computing the hash of their ID and accessing the corresponding index in the table.

Collision in Hash Table

Collision in a hash table occurs when two or more keys are hashed to the same index in the hash table. Since each index can ideally hold only one value, collisions need to be resolved to maintain the integrity and efficiency of the hash table.

Why Do Collisions Occur?

1. **Limited Array Size:** The hash table has a finite number of slots, so multiple keys may map to the same index when the hash function produces the same result.
2. **Hash Function Limitations:** Even with a good hash function, it's impossible to guarantee unique hashes for every key, especially when the key space is much larger than the table size.

Collision Resolution Techniques

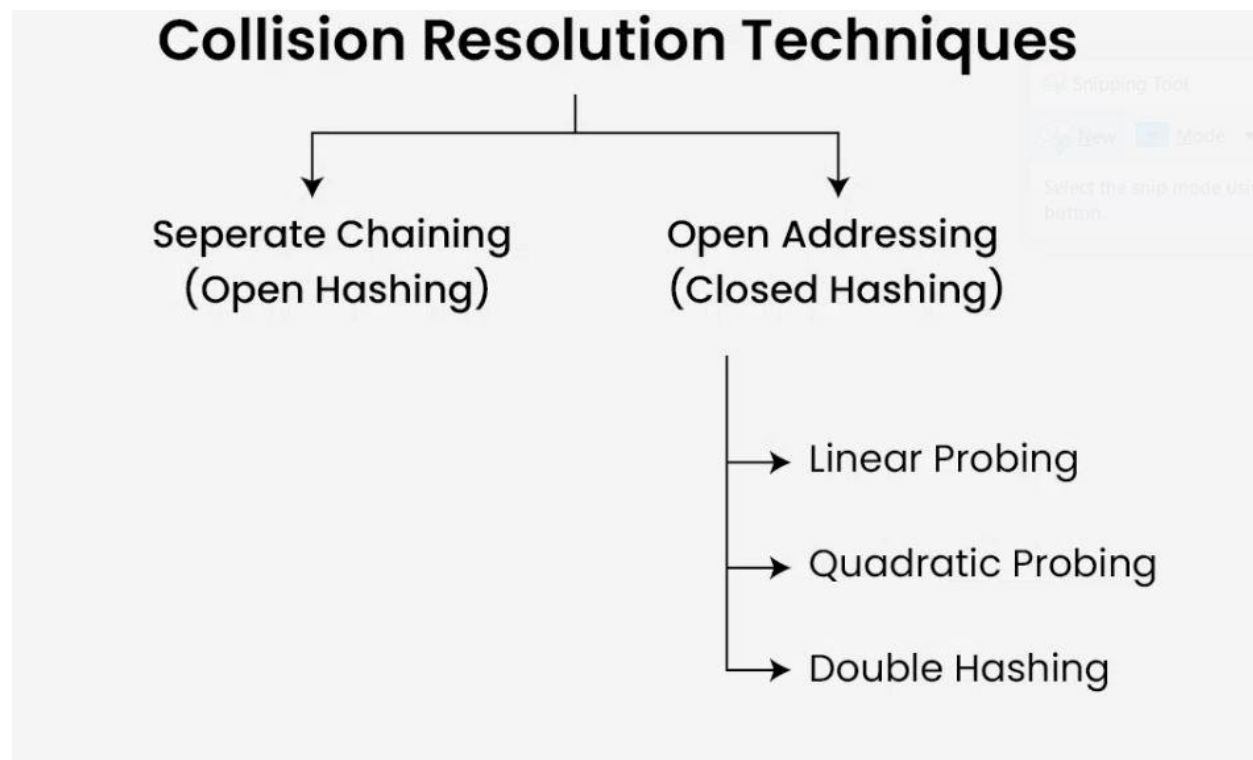
In **Hashing**, hash functions were used to generate hash values. The hash value is used to create an index for the keys in the hash table. The hash function may return the same hash value for two or more keys. When two or

more keys have the same hash value, a **collision** happens. To handle this collision, we use **Collision Resolution Techniques**.

Collision Resolution Techniques

There are mainly two methods to handle collision:

1. Open Hashing / Separate Chaining
2. Closed Hashing / Open Addressing



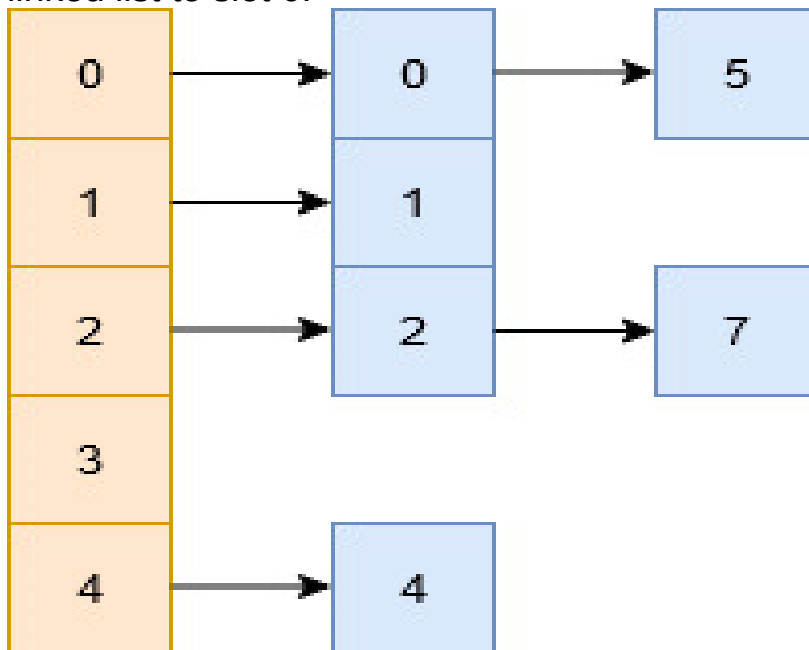
1. Open Hashing (Separate chaining)

Collisions are resolved using a list of elements to store objects with the same key together.

- Suppose you wish to store a set of numbers = {0,1,2,4,5,7} into a hash table of size 5.
- Now, assume that we have a hash function H , such that $H(x) = x \% 5$
- So, if we were to map the given data with the given hash function we'll get the corresponding values

- $H(0) \rightarrow 0\%5 = 0$
- $H(1) \rightarrow 1\%5 = 1$
- $H(2) \rightarrow 2\%5 = 2$
- $H(4) \rightarrow 4\%5 = 4$
- $H(5) \rightarrow 5\%5 = 0$
- $H(7) \rightarrow 7\%5 = 2$

- Clearly 0 and 5, as well as 2 and 7 will have the same hash value, and in this case we'll simply append the colliding values to a list being pointed by their hash keys. Now, separate chaining will handle collision by creating a linked list to slot 0.



2) Open Addressing

In open addressing, all elements are stored in the hash table itself. Each table entry contains either a record or NIL. When searching for an element, we examine the table slots one by one until the desired element is found or it is clear that the element is not in the table.

2.a) Linear Probing

In linear probing, the hash table is searched sequentially that starts from the original location of the hash. If in case the location that we get is already occupied, then we check for the next location.

Algorithm:

1. Calculate the hash key. i.e. **key = data % size**
2. Check, if **hashTable[key]** is empty
 - store the value directly by **hashTable[key] = data**
3. If the hash index already has some value then
 - check for next index using **key = (key+1) % size**
4. Check, if the next index is available hashTable[key] then store the value. Otherwise try for next index.
5. Do the above process till we find the space.

Linear Probing Example

Insert (76)	Insert (93)	Insert (40)	Insert (47)	Insert (10)	Insert (55)
$76\%7 = 6$	$93\%7 = 2$	$40\%7 = 5$	$47\%7=5$	$10\%7=3$	$55\%7=6$
0 1 2 3 4 5 6 76	0 1 2 3 4 5 6 93 76	0 1 2 3 4 5 6 93 40 76	0 1 2 3 4 5 6 47 93 40 76	0 1 2 3 4 5 6 47 93 10 40 76	0 1 2 3 4 5 6 47 55 93 10 40 76

2. Quadratic Probing

Quadratic probing is a method with the help of which we can solve the problem of clustering that was discussed above. This method is also known as the **mid-square** method. In this method, we look for the i^2 'th slot in the i th iteration. We always start from the original hash location. If only the location is occupied then we check the other slots.

How Quadratic Probing is done?

Let $\text{hash}(x)$ be the slot index computed using the hash function.

- If the slot $\text{hash}(x) \% S$ is full, then we try $(\text{hash}(x) + 1^2) \% S$.
- If $(\text{hash}(x) + 1^2) \% S$ is also full, then we try $(\text{hash}(x) + 2^2) \% S$.
- If $(\text{hash}(x) + 2^2) \% S$ is also full, then we try $(\text{hash}(x) + 3^2) \% S$.
- This process is repeated for all the values of i until an empty slot is found.

Example

Let us consider table Size = 7, hash function as $\text{Hash}(x) = x \% 7$ and collision resolution strategy to be $f(i) = i^2$. Insert = 22, 30, and 50.

Quadratic Probing (Open Addressing)



Slot

0	
1	
2	
3	
4	
5	
6	

Step 01

Empty hash table with range of hash values from **0 to 6** according to the hash function provided.

Quadratic Probing (Open Addressing)



Slot

0	
1	22
2	
3	
4	
5	
6	

Step 02

The first key to be inserted is **22** which is mapped to slot 1 ($22\%7=1$)

Quadratic Probing (Open Addressing)



Slot

0	
1	22
2	30
3	
4	
5	
6	

Step 03

The next key is 30 which is mapped to slot 2 ($30\%7=2$)

Quadratic Probing (Open Addressing)



Slot

0	
1	22
2	30
3	
4	
5	50
6	

← $1+0$
← $1+1^2$
← $1+2^2$

Step 04

The next key is 50 which is mapped to slot 1 ($50\%7=1$) but slot 1 is already occupied. So, we will search slot $1+1^2$, i.e. $1+1 = 2$. Again slot 2 is occupied, so we will search cell $1+2^2$, i.e. $1+4 = 5$,

3. Double Hashing

Double hashing is a collision resolution technique used in hash tables. It works by using two hash functions to compute two different hash values for a given key. The first hash function is used to compute the initial hash value, and the second hash function is used to compute the step size for the probing sequence.

Double hashing has the ability to have a low collision rate, as it uses two hash functions to compute the hash value and the step size. This means that the probability of a collision occurring is lower than in other collision resolution techniques such as linear probing or quadratic probing.

However, double hashing has a few drawbacks. First, it requires the use of two hash functions, which can increase the computational complexity of the insertion and search operations. Second, it requires a good choice of hash functions to achieve good performance. If the hash functions are not well-designed, the collision rate may still be high.

Double hashing is a technique that reduces clustering in an optimized way. In this technique, the increments for the probing sequence are computed by using another hash function. We use another hash function $\text{hash2}(x)$ and look for the $i * \text{hash2}(x)$ slot in the i th rotation.

Double hashing can be done using :

$(\text{hash1}(\text{key}) + i * \text{hash2}(\text{key})) \% \text{TABLE_SIZE}$

Here $\text{hash1}()$ and $\text{hash2}()$ are hash functions and TABLE_SIZE is size of hash table.

(We repeat by increasing i when collision occurs)

let $\text{hash}(x)$ be the slot index computed using hash function.

*If slot $\text{hash}(x) \% S$ is full, then we try $(\text{hash}(x) + 1 * \text{hash2}(x)) \% S$*

*If $(\text{hash}(x) + 1 * \text{hash2}(x)) \% S$ is also full, then we try $(\text{hash}(x) + 2 * \text{hash2}(x)) \% S$*

*If $(\text{hash}(x) + 2 * \text{hash2}(x)) \% S$ is also full, then we try $(\text{hash}(x) + 3 * \text{hash2}(x)) \% S$*

Double Hashing Algorithm for Inserting an Element

1. Set $\text{index} = H(K)$; $\text{offset} = H2(K)$
2. If table location index already contains the key, no need to insert it. Done!
3. Else if table location index is empty, insert key there. Done!
4. Else collision. Set $\text{index} = (\text{index} + \text{offset}) \bmod M$.

5. If $\text{index} == H(K)$, table is full! (Throw an exception, or enlarge table.)
Else go to step 2.

Let's implement the above algorithm in a Java program.

The idea behind double hashing is fairly simple,

1. Take the key you want to store on the hash-table.
2. Apply the first hash function $h_1(\text{key})$ over your key to get the location to store the key.
3. If the location is empty, place the key on that location.
4. If the location is already filled, apply the second hash function $h_2(\text{key})$ in combination with the first hash function $h_1(\text{key})$ to get the new location for the key.

Time Complexity of Double Hashing

- Best Case: In the best case, no collision would occur and thus, the time complexity will be given by **$O(1)$** .
- Average Case: The average case time complexity is also given by **$O(1)$** . However, its proof is too complex to be discussed here.
- Worst Case: In the worst case, we have to probe over all **N** elements; thus the time complexity is given by **$O(N)$** .

Space Complexity of Double Hashing

Hash tables usually have a maximum capacity of some constant multiplied by **N** (the constant is predetermined and **N** is the total number of elements.) Thus the space complexity of double hashing is **$O(N)$** .

Advantages of Double Hashing

- No extra space is required.
- Primary Clustering does not occur.
- Secondary Clustering also does not occur.

- It can be used with different types of hash functions.
- It reduces the clustering effect of keys and decreases the chance of keys being stored in consecutive buckets.

Disadvantages of Double Hashing

- Poor cache performance.
- It's a little complicated because it uses two hash functions.
- It has a bit harder implementation.
- It consumes more memory than other collision resolution techniques.
- It may not be the best choice for small tables.

Problem Statement:

Insert the keys 79, 69, 98, 72, 14, 50 into the **Hash Table of size 13**. Resolve all collisions using Double Hashing where first hash-function is $h_1(k) = k \bmod 13$ and second hash-function is $h_2(k) = 1 + (k \bmod 11)$

Note: Where 11 is the nearest Prime Number of Hash Table size.

Solution:

Initially, all the hash table locations are empty. We pick our **first key = 79** and apply $h_1(k)$ over it,

$h_1(79) = 79 \% 13 = 1$, hence key 79 hashes to the 1st location of the hash table. But before placing the key, we need to make sure the 1st location of the hash table is empty. In our case it is empty and we can easily place key 79 in there.

Second key = 69, we again apply $h_1(k)$ over it, $h_1(69) = 69 \% 13 = 4$, since the 4th location is empty we can place 69 there.

Third key = 98, we apply $h_1(k)$ over it, $h_1(98) = 98 \% 13 = 7$, 7th location is empty so 98 placed there.

Fourth key = 72, $h_1(72) = 72 \% 13 = 7$, now this is a collision because the 7th location is already occupied, we need to resolve this collision using double hashing.

$$h_{new} = [h_1(72) + i * h_2(72)] \% 13$$

$$= [7 + 1 * (1 + 72 \% 11)] \% 13$$

$$= 1$$

Location **1st** is already occupied in the hash-table, hence we again had a collision. Now we again recalculate with $i = 2$

$$h_{new} = [h_1(72) + i * h_2(72)] \% 13$$

$$= [7 + 2 * (1 + 72 \% 11)] \% 13$$

$$= 8$$

Location 8th is empty in hash-table and now we can place key 72 in there.

Fifth key = 14, $h_1(14) = 14 \% 13 = 1$, now this is again a collision because 1st location is already occupied, we now need to resolve this collision using double hashing.

$$h_{new} = [h_1(14) + i * h_2(14)] \% 13$$

$$= [1 + 1 * (1 + 14 \% 11)] \% 13$$

$$= 5$$

Location 5th is empty and we can easily place key 14 there.

Sixth key = 50, $h_1(50) = 50 \% 13 = 11$, 11th location is already empty and we can place our key 50 there.

Load Factor

The Load factor is a measure that decides when to **increase** the HashMap capacity to maintain the get() and put() operation complexity of **$O(1)$** . The default load factor of HashMap is **0.75f** (75% of the map size).

Problem

The problem is, keeping the bucket size fixed (i.e., 16), we keep on increasing the total number of items in the map that disturbs time complexity.

Solution

When we increase the total number of buckets, total items in each bucket starts increasing. Now we are able to keep the constant number of items in each bucket and maintain the time complexity of $O(1)$ for get() and put() operation.

How hashing works?

For insertion of a key(K) – value(V) pair into a hash map, 2 steps are required:

1. K is converted into a small integer (called its hash code) using a hash function.
2. The hash code is used to find an index ($\text{hashCode} \% \text{arrSize}$) and the entire linked list at that index (Separate chaining) is first searched for the presence of the K already.
3. If found, it's value is updated and if not, the K-V pair is stored as a new node in the list.

Load Factor Example

Let's understand the load factor through an example.

If we have the initial capacity of HashTable = 16.

We insert the first element and now check if we need to increase the size of the HashTable capacity or not.

It can be determined by the formula:

Size of hashmap (m) / number of buckets (n)

In this case, the size of the hashmap is 1, and the bucket size is 16.
So, $1/16=0.0625$. Now compare this value with the default load factor.

$$0.0625 < 0.75$$

So, no need to increase the hashmap size.

We do not need to increase the size of hashmap up to 12th element, because

$$12/16=0.75$$

As soon as we insert the 13th element in the hashmap, the size of hashmap is increased because:

$$13/16=0.8125$$

Which is greater than the default hashmap size.

$$0.8125 > 0.75$$

Now we need to increase the hashmap size.

Rehashing:

Rehashing is the process of increasing the size of a hashmap and redistributing the elements to new buckets based on their new hash values. It is done to improve the performance of the hashmap and to prevent collisions caused by a high load factor.

When a hashmap becomes full, the load factor (i.e., the ratio of the number of elements to the number of buckets) increases. As the load factor increases, the number of collisions also increases, which can lead to poor performance. To avoid this, the hashmap can be resized and the elements

can be rehashed to new buckets, which decreases the load factor and reduces the number of collisions.

During rehashing, all elements of the hashmap are iterated and their new bucket positions are calculated using the new hash function that corresponds to the new size of the hashmap. This process can be time-consuming but it is necessary to maintain the efficiency of the hashmap.

Rehashing Steps in Data Structure

The rehashing process contains the following steps:

1. First of all check the load factor of the insertion of each new element to the HashMap.
2. If the load factor is greater than its default value i.e. 0.75, then implement rehashing technique.
3. Create a new array of the size double than the size of the previous array.
4. Iterate over the previous array elements and on each element make a call to the insert() method. The insert() method inserts all the elements in the newly created array (larger in size).

Why rehashing?

Rehashing is needed in a hashmap to prevent collision and to maintain the efficiency of the data structure.

As elements are inserted into a hashmap, the load factor (i.e., the ratio of the number of elements to the number of buckets) increases. If the load factor exceeds a certain threshold (often set to 0.75), the hashmap becomes inefficient as the number of collisions increases. To avoid this, the hashmap can be resized and the elements can be rehashed to new buckets, which decreases the load factor and reduces the number of collisions. This process is known as rehashing.

Rehashing can be costly in terms of time and space, but it is necessary to maintain the efficiency of the hashmap.

This means if we had an array of size 100 earlier, and once we have stored 75 elements into it (given it has Load Factor=0.75), then when we need to store the 76th element, we double its size to 200.

